

Национальный исследовательский университет ИТМО
Факультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Отчет по практическому заданию №1
по курсу «Языки программирования»

Вариант: 2

Выполнил студент группы Р4119

(подпись)

Н.Ю. Ковалев

Руководитель

(подпись)

Ю. Д. Кореньков

Санкт-Петербург

2025

Цели

- Собрать модуль синтаксического анализа исходного языка варианта, реализованный на базе Tree-sitter, и вывести синтаксическое дерево, пригодное для визуального просмотра.
- Демонстрировать работу модуля на примерах, покрывающих все конструкции варианта (подпрограммы, управление потоком, литералы, массивы, выражения и вызовы функций).

Задачи

1. Подготовить спецификацию языка (файл `Lab1/src/grammar.json`) с описанием конструкций функций, переменных, ветвлений, циклов, литералов и выражений разной сложности.
2. Сгенерировать из неё парсер C/C++ с использованием Tree-sitter (`Lab1/src/parser.c`) и встроить его в модуль `ast_dump`.
3. Реализовать CLI `ast_dump`, принимающий текст и опции, возвращающий иерархическое дерево разбора в формате DOT (`ast_dump.c`).
4. Подготовить тестовый прогон (`Lab1/generate_trees.sh` и `run_lab1.sh`), который собирает модуль и последовательно парсит все образцы из `Lab1/examples`, сохраняя `.dot` и `.pdf`.
5. Описать структуры результатов разбора и доп. обработку, формирующую финальный AST для выдачи результатов и тестовых отчётов.

Структуры данных результата (задание 3a)

Результатом синтаксического анализа является корневой `TSNode` и его дети. `ast_dump` сохраняет `TypeFilter` (массив `types[]`), используемый для фильтрации отображаемых узлов, и каждой вершине сопоставляет тип, диапазон байт и текст токена (`identifier`, `dec`, `str`, `char`, `bool` и др.). Функции `is_wrapper_type`/`is_trivial_wrapper` исключают промежуточные обёртки, не нарушая структуры AST. Эта структура потом используется при построении DOT — каждый узел получает метку вида `id: имя / num: значение` и поддержку окраски.

Описание работы

Модуль строится вокруг сгенерированного Tree-sitter-движка `v2lang_test` (`Lab1/src/parser.c`), который компилируется вместе с `vendor/tree-sitter/lib` и используется в исполняемом файле `ast_dump`. CLI ожидает два пути (`<вход>.txt` и `<выход>.dot`) и необязательный флаг `--only=тип,тип`, фильтрующий видимые узлы дерева. Программа читает файл с помощью `read_file`, создаёт `TSParser`, устанавливает язык `tree_sitter_v2lang_test()`, парсит строку и рекурсивно обходит дерево через `dump_node_dot`, пропуская «обёртки» (`is_trivial_wrapper`) и раскрашивая важные конструкции (например, `assignment`, `if_statement`, `funcDef`, `varDecl`) для наглядности.

Дополнительная обработка результата синтаксического анализа (задание 4b)

Парсер выдаёт `TSNode`, однако для получения графического AST потребовалась постобработка: `dump_node_dot` обходит дерево, пропуская узлы-«обёртки», извлекает содержимое токенов (с помощью `print_label_escaped`) и добавляет атрибуты (`style`, `fillcolor`, `shape`) для ключевых типов (`assignment`, `if_statement`, `funcDef`, `varDecl`). `TypeFilter` позволяет показывать только выбранные типы, а при отображении листьев выводятся краткие метки (`id: x`, `num: 42`), чтобы граф был читабельным и соответствовал структуре AST.

Набор тестов представлен скриптом `Lab1/generate_trees.sh`: при первом запуске он собирает `ast_dump` командой `clang ast_dump.c Lab1/src/parser.c ...`, а затем парсит каждый `.txt` из `Lab1/examples`, сохраняя `.dot` и (если установлен `Graphviz`) `.pdf`. Скрипт `run_lab1.sh` просто вызывает `generate_trees.sh` и служит точкой входа, обернув сборку в безопасный `bash`-контекст.

Аспекты реализации

- **Спецификация языка.** В `Lab1/src/grammar.json` описаны все требуемые конструкции: `funcDef` с `funcSignature` и списком аргументов, блоки `begin ... end;`, ветвления `if ... then ... else`, циклы `while ... do`, `repeat ... while/until`, декларации переменных `var` с `idList`, литералы (`dec`, `hex`, `bits`, `str`, `char`, `bool`), выражения с логическими, битовыми и арифметическими операторами, одномерные массивы через `array [ , ] of typeRef`, вызовы функций через `postfix ( exprList? )` и доступ к массивам `postfix [ exprList? ]`. Все уровни выражений оформлены как `logical_or`, `logical_and`, `bitwise_*`, `relational`, `shift`, `add`, `mul`, `unary`, что обеспечивает правильный приоритет операций.
- **Структуры данных результата.** Парсер возвращает корневой `TSNode`, из которого текущий модуль `ast_dump` строит DOT-граф: для каждого узла сохраняется тип, диапазон байт и текстовые значения токенов (`identifier`, `dec`, `str`, `char` и т.д.). `TypeFilter` модуля позволяет исключать лишние типы по списку, а `dump_node_dot` рекурсивно добавляет узлы в граф, сохраняя связь с родителем.
- **Дополнительная обработка.** В `dump_node_dot` узлы-«обёртки» (например, `_expr`, `logical_or` и т.п.) пропускаются, если они содержат ровно один именованный ребёнок; таким образом дерево не дублирует промежуточные уровни. Метка узла собирается с учётом короткого имени и лексем (например, `id: x`, `num: 42`), а для ключевых конструкций добавляются атрибуты `style`, `fillcolor`, `shape`, чтобы при визуализации выделять функции, объявления, присваивания и условные переходы.

Ключевые правила грамматики `Lab1/src/grammar.json`

- `funcSignature / funcDef` — `method identifier ( argList? ) [: typeRef]` и тело `body` или `;`.
- `body` содержит повтор `var` с `varDecl ( idList [: typeRef] ; )` и основной `block`.
- `typeRef` описывает базовые типы, идентификаторы и `array [ , ... ] of typeRef` для массивов.
- `statement` раскрывает `if_statement`, `while_statement`, `do_statement`, `break_statement`, `return_statement`, `assignment`, `expr_stmt` и блоки.

- `expr` оформлен как цепочка приоритетов (`logical_or` → `logical_and` → `bitwise_*` → `relational` → `shift` → `add` → `mul` → `unary` → `postfix`).
- `postfix` поддерживает вызовы `postfix ( exprList? )` и индексирование `postfix [ exprList? ]`.
- `literal` объединяет `dec`, `hex`, `bits`, `str`, `char`, `bool`, а `identifier` описывается как `[a-zA-Z_][a-zA-Z_0-9]*`; `extras` — пробелы и `comment`.

Фрагмент из Grammar

```
"funcSignature": {
  "type": "SEQ",
  "members": [
    { "type": "SYMBOL", "name": "identifier" },
    { "type": "STRING", "value": "(" },
    {
      "type": "CHOICE",
      "members": [
        {
          "type": "SEQ",
          "members": [
            {
              "type": "CHOICE",
              "members": [
                { "type": "SYMBOL", "name": "argList" },
                { "type": "BLANK" }
              ]
            }
          ]
        },
        { "type": "BLANK" }
      ]
    },
    { "type": "STRING", "value": ")" },
    {
      "type": "CHOICE",
      "members": [
        {
          "type": "SEQ",
          "members": [
            { "type": "STRING", "value": ":" },
            { "type": "SYMBOL", "name": "typeRef" }
          ]
        },
        { "type": "BLANK" }
      ]
    }
  ]
}
```

этот фрагмент показывает, как задания меток аргументов и опционального типа, а `funcDef` затем подключает `body` или завершается `; .`

```
TSParser *parser = ts_parser_new();
if (!ts_parser_set_language(parser, tree_sitter_v2lang_test())) {
    fprintf(stderr, "Failed to set language.\n");
    return 3;
}
TSTree *tree = ts_parser_parse_string(parser, NULL, source, (uint32_t)len);
dump_node_dot(out, ts_tree_root_node(tree), -1, &next_id, source, &filter);
```

Примеры исходных текстов (задание 5с)

- `Lab1/examples/all_structures_and_expr.txt` демонстрирует объявления `var`, `begin...end`, `if/else`, `while/ repeat`, одномерные массивы и смешанные выражения (арифметика, битовые, логика); соответствующие DOT/PDF отображают каждую ветвь и блок.
- `Lab1/examples/functions.txt`, `fib_recur.txt`, `echo.txt` покрывают объявления `method`, параметры и возвращаемые значения, рекурсию, вызовы функций, литералы (строки, символы, числа) и вывод `outd`; их деревья отражают `funcDef`, `funcSignature` и `exprList`.
- `Lab1/examples/type_correct.txt` и `type_errors.txt` проверяют устойчивость при корректных/ошибочных декларациях и присваиваниях: корректный файл генерирует дерево со всеми конструкциями, а `type_errors` даёт диагностические сообщения в `stderr`, не нарушая генерацию остальных DOT-файлов.
- Дополнительные файлы (`my_test.txt`, `fib_return.txt`, `type_errors.txt`) используются для проверки литералов, индексации массивов и вложенных выражений; каждая проверка сопровождается соответствующим DOT/PDF/CSV.

Для каждого примера из `Lab1/examples` сохраняется `*.dot` и (при наличии Graphviz) `*.pdf` в `Lab1/examples/`, благодаря `generate_trees.sh`.

Результаты

- Собран исполняемый модуль `ast_dump` (компиляция `ast_dump.c`, `Lab1/src/parser.c` и `vendor/tree-sitter/lib`).
- Сгенерированы деревья DOT (и PDF при наличии `dot`) для всех примеров `Lab1/examples/*.txt`, включая `all_structures_and_expr`, `functions`, `fib_recur`, `echo`, `type_correct`, `type_errors` и другие (см. файлы `*.dot`, `*.pdf`).
- Тестовый прогон `run_lab1.sh/generate_trees.sh` вызвал парсер для каждой тестовой программы и вёл вывод `Parsing ... -> ... .dot`, фиксируя успешность анализа и обработку опций фильтра типов. Ошибки парсинга (например, неверный синтаксис) выводятся в `stderr` из `ast_dump`.

Результаты тестирования

Скрипт `Lab1/generate_trees.sh` (запускаемый через `run_lab1.sh`) прогоняет парсер по каждому файлу из `Lab1/examples/*.txt` и сохраняет результат в соответствующий `.dot` (и `.pdf`, если установлен Graphviz). Ниже сводка ключевых примеров и проверяемых конструкций:

Пример	Покрываемые конструкции	Что подтверждает результат
<code>all_structures_and_expr.txt</code>	<code>var</code> -объявления, <code>begin...end</code> , <code>if/else</code> , <code>while</code> , <code>repeat</code> , массивы, смешанные выражения (арифметика, логика, битовые операции)	DOT и PDF показывают полную иерархию, демонстрируя корректную расстановку приоритетов и вложенных блоков
<code>functions.txt</code>	<code>method</code> -определения с аргументами/типом возврата, вызовы функций	Узлы <code>funcDef</code> , <code>funcSignature</code> , <code>exprList</code> явно описывают параметры, возвращаемое значение и аргументы вызовов
<code>fib_recur.txt</code>	Рекурсивные вызовы, ветвления, арифметика	Результирующее DOT-дерево отражает рекурсивную структуру и позволяет отслеживать пути выполнения
<code>echo.txt</code>	Ввод/вывод, строковые и символьные литералы, простые вызовы процедур	Литералы маркируются как <code>str</code> , <code>char</code> , а вызов <code>print</code> отображается в <code>postfix</code>
<code>type_correct.txt</code> / <code>type_errors.txt</code>	Корректная и некорректная семантика объявлений/присваиваний	При корректном коде дерево строится полностью, при ошибках <code>ast_dump</code> пишет диагностические сообщения в <code>stderr</code>
<code>my_test.txt</code> , <code>exmpl.txt</code>	Дополнительные сценарии: <code>break</code> , комбинированные выражения, массивы с аргументами	DOT-файлы подтверждают устойчивость разбора к разнообразию синтаксических конструкций

Эти результаты показывают, что модуль успешно разбирает все требуемые элементы языка и визуализирует их структуру, а тестовый скрипт фиксирует успех для каждого примера.

Выводы

- Достигнута цель: реализован модуль синтаксического анализа языка варианта, возвращающий дерево разбора в читаемом графовом формате и покрывающий все требуемые конструкции (функции, ветвления, циклы, объявления, литералы, выражения, массивы, вызовы).
- Тестовая программа через `generate_trees.sh` позволяет удобно регенерировать `.dot` / `.pdf` и убедиться, что парсер устойчив к разным сценариям; визуальные деревья подтверждают структуру и правильность разбора.
- Работа с Tree-sitter дала опыт описания грамматики JSON-форматом, генерации C-парсера и последующей постобработки AST (фильтрация, раскраска, точные метки). Далее можно расширить генерацию отчётов, добавить сериализацию дерева или анализ семантики.